## Tema: Las 4 P de la Gestión de Proyectos de Software

**(Basado en Pressman – Ingeniería de Software)**

---

### 0️⃣ Introducción: La Gestión como Pilar del Éxito en Software

Cuando nos adentramos en el mundo de la ingeniería de software, la imagen mental predominante suele ser la del programador resolviendo problemas complejos, diseñando arquitecturas elegantes o realizando pruebas exhaustivas. Si bien estas actividades técnicas son el núcleo de la construcción, Roger Pressman, en su obra seminal "Ingeniería del Software: Un Enfoque Práctico", plantea una premisa fundamental que redefine las prioridades:

**"La mayoría de los fracasos en proyectos de software no se deben a errores de programación, sino a fallas de gestión."**

Esta afirmación es un llamado de atención. Gestionar un proyecto de software es mucho más que "organizar tareas" en una lista. Es una disciplina compleja que implica coordinar un ecosistema dinámico compuesto por:

- **Personas:** Con sus habilidades, motivaciones, conflictos y necesidades de comunicación.
- **Objetivos:** A menudo cambiantes, ambiguos o contradictorios entre los stakeholders.
- **Restricciones:** De tiempo, presupuesto, tecnología, recursos y normativas legales.
- **Incertidumbre:** Asociada a la tecnología, a la estimación y a los riesgos no identificados.
- **Cambio:** La única constante en el software, que debe ser gestionado para que no se convierta en caos.

Para navegar esta complejidad, Pressman propone un modelo conceptual simple pero profundamente poderoso: **Las 4 P de la Gestión de Proyectos de Software**. Este marco no es una lista de verificación estática, sino un sistema de dimensiones interconectadas que, analizadas en conjunto, proporcionan una visión holística del proyecto.

**Las 4 P son:**

1. **People (Personas)**
2. **Product (Producto)**
3. **Process (Proceso)**
4. **Project (Proyecto)**

El orden en que se presentan no es casual. Establece una jerarquía de dependencia, donde el factor humano es la base sobre la que se construye todo lo demás.

---

## 1️⃣ PEOPLE – PERSONAS: El Corazón del Proyecto

**Idea Central: El Factor Humano como Activo Primordial**

El software es, ante todo, un producto intelectual. Es el resultado del pensamiento, la creatividad y el esfuerzo de personas. Ninguna herramienta, metodología o tecnología por sí sola puede construir software de valor. Por lo tanto, la gestión comienza y termina en las personas. Pressman enfatiza con vehemencia: **"Si el equipo no funciona correctamente, el proyecto no funcionará, aunque el proceso sea excelente"**.

**¿Qué Implica Gestionar Personas?**

Va mucho más allá de la asignación de tareas. Es una función de liderazgo y desarrollo humano que abarca:

- **Formación de Equipos Equilibrados:** Construir grupos de trabajo que no solo tengan las habilidades técnicas necesarias, sino que también posean diversidad de pensamiento y una dinámica interpersonal saludable.
- **Selección de Perfiles Adecuados:** Reconocer que no todos los roles son para todos. Un excelente programador puede no ser un buen arquitecto, y un brillante analista puede no tener las habilidades para liderar un equipo en momentos de crisis.
- **Fomento de la Comunicación:** Crear un entorno donde la información fluya de manera abierta, respetuosa y eficaz, minimizando ruidos y malentendidos.
- **Resolución de Conflictos:** El conflicto es inevitable en entornos de alta presión. Gestionarlo de forma proactiva es crucial para evitar que se convierta en una fuerza destructiva.
- **Motivación:** Comprender qué impulsa a cada miembro del equipo (reconocimiento, autonomía, aprendizaje, etc.) y alinear las tareas con esos motivadores.
- **Mantenimiento de la Estabilidad:** Proteger al equipo de la rotación excesiva y la desintegración, que son costosas y desestabilizadoras.

**Elementos Clave en la Gestión de PERSONAS**

1. **Competencia Técnica y Estratégica:**
    - **Descripción:** No todos los desarrolladores poseen el mismo nivel de habilidad ni las mismas fortalezas. Algunos son especialistas en front-end, otros en bases de datos, algunos son pensadores abstractos ideales para la arquitectura, y otros son meticulosos y perfectos para el aseguramiento de la calidad.
    - **Ejemplo de Falla:** Un proyecto que requiere un diseño arquitectónico robusto y escalable asigna esa responsabilidad a un desarrollador junior con poca experiencia en sistemas distribuidos. El resultado probable será un diseño frágil, con un alto acoplamiento y difícil de mantener, que comprometerá todo el desarrollo futuro y generará una deuda técnica masiva.

- **Gestión Adecuada:** El líder de proyecto debe identificar las fortalezas y debilidades de cada individuo. Utilizar herramientas como la matriz de habilidades para mapear quién es el más adecuado para cada tarea crítica. Esto no solo optimiza el resultado, sino que también contribuye al desarrollo profesional de los miembros del equipo al asignarles retos acordes a su nivel.

2. **Comunicación: El Tejido Conectivo del Equipo:**
    - **Descripción:** La comunicación es el lubricante que permite que el motor del equipo funcione sin fricción. Su complejidad crece de manera exponencial con el tamaño del equipo. La fórmula [n*(n-1)]/2 (donde 'n' es el número de personas) nos da el número de canales de comunicación potenciales.
        - Con **4 personas**: 6 canales. La comunicación puede ser informal y autogestionada.
        - Con **8 personas**: 28 canales. La complejidad se cuadruplica. La comunicación informal colapsa.
    - **Conclusión Clave:** Añadir más personas a un proyecto que está retrasado no siempre acelera las cosas (Ley de Brooks). De hecho, puede empeorarlas al aumentar la carga de comunicación y el tiempo de integración.
    - **Rol de la Gestión:** El gestor debe establecer **canales formales de comunicación**. Esto incluye definir protocolos claros para:
        - **Reuniones:** Diarias (Daily Stand-up), de planificación (Sprint Planning), de revisión (Review) y de retrospectiva (Retrospective).
        - **Documentación:** Qué se documenta, con qué nivel de detalle y dónde se almacena (ej. Wiki, repositorio compartido).
        - **Herramientas:** Definir el uso de Slack/Teams para comunicación asíncrona, Jira/Trello para el seguimiento de tareas, y Git para la gestión del código, evitando la dispersión de información.
3. **Liderazgo: El Timón en la Tormenta:**
    - **Descripción:** El líder de proyecto no es simplemente un "programador senior" con más responsabilidades. Es un rol organizacional distinto, cuyo objetivo es guiar al equipo hacia el éxito. Su enfoque principal son las personas y el proyecto, no el código.
    - **Responsabilidades Críticas:**
        - **Toma de Decisiones Estratégicas:** Decidir entre opciones tecnológicas, priorizar funcionalidades, aceptar o mitigar riesgos.
        - **Mediación de Conflictos:** Actuar como un árbitro neutral cuando surgen disputas técnicas o personales, buscando soluciones de consenso.

        - **Definición de Prioridades:** Traducir los objetivos del negocio en prioridades claras para el equipo, protegiéndolo de la volatilidad externa.
        - **Gestión de la Presión Externa:** Actuar como un "pararrayos" ante la presión de los stakeholders, filtrando demandas irreales y comunicando el progreso de manera efectiva.
        - **Mantenimiento de la Cohesión:** Fomentar un sentido de propósito común y de pertenencia, celebrando los logros y apoyando al equipo en los momentos difíciles.
    - **Consecuencias de un Liderazgo Débil:** Falta de dirección clara, fragmentación del equipo en subgrupos con agendas propias, retrasos por falta de toma de decisiones, y desmotivación general.

4. **Estabilidad del Equipo: Preservando el Conocimiento:**
    - **Descripción:** La rotación de personal es uno de los riesgos más graves y costosos en el desarrollo de software. Cuando un miembro clave del equipo, especialmente uno con mucho conocimiento del dominio o de la arquitectura del sistema, abandona el proyecto, el impacto es multidimensional.
    - **Impactos Negativos de la Rotación:**
        - **Pérdida de Conocimiento Tácito:** Se pierde la comprensión profunda de las decisiones de diseño, las sutilezas del código y las relaciones con los stakeholders, que no está escrita en ningún documento.
        - **Retraso en la Integración:** El nuevo miembro requerirá un tiempo de "curva de aprendizaje" para ponerse al día, durante el cual su productividad será baja y requerirá tiempo de otros miembros para ser mentorizado.
        - **Aumento del Riesgo:** Aumenta la probabilidad de introducir nuevos errores al modificar código que no se comprende completamente y se pierde la memoria de por qué se hicieron ciertas cosas.
    - **Conexión con Gestión de Riesgos:** La estabilidad del equipo es un aspecto central de la gestión de riesgos humanos. Un plan de mitigación puede incluir la documentación rigurosa, el pair programming (programación en pareja) para transferir conocimiento, y un entorno de trabajo que fomente la retención del talento.

**Conclusión Fuerte sobre PEOPLE**

El software es un producto intelectual colectivo, un "cerebro de colmena" materializado en código. Descuidar la gestión humana no es solo un riesgo; es garantizar la erosión de los cimientos del proyecto. Por ello, las personas constituyen el primer pilar y el más fundamental de las 4 P.

---

## 2️⃣ PRODUCT – PRODUCTO: Definiendo el "Qué"

**Idea Central: Comprender para Construir**

Antes de cualquier planificación, estimación o asignación de recursos, debemos responder a la pregunta fundamental: **¿Qué vamos a construir?** Pressman lo plantea con claridad: "No se puede estimar ni planificar lo que no se comprende". La definición del producto es el faro que guía todas las actividades posteriores.

**¿Qué Implica Definir el PRODUCTO?**

Definir el producto es un ejercicio de análisis, negociación y especificación que va mucho más allá de una simple lista de deseos. Implica establecer de manera rigurosa:

- **Alcance (Scope):** Los límites del sistema.
- **Objetivos:** El "para qué" del sistema, su valor de negocio.
- **Funcionalidades:** Las capacidades específicas que el sistema ofrecerá al usuario.
- **Requerimientos No Funcionales:** Los atributos de calidad del sistema.
- **Restricciones:** Limitaciones técnicas, legales, de negocio o de plataforma.
- **Contexto Organizacional:** Cómo encaja el sistema en la estructura y procesos de la organización.

**Profundizando en los Elementos Clave**

1. **Alcance: El Arte de Decir "No"**
    - **Descripción:** El alcance es la frontera que separa lo que el sistema **hará** de lo que **no hará**. Es un contrato implícito entre el equipo de desarrollo y el cliente/propietario del producto. Definirlo con precisión es crucial para evitar el crecimiento incontrolado del proyecto.
    - **Ejemplo Detallado: Sistema de Gestión Bibliotecaria.**
        - **Dentro del Alcance (Incluye):**
            - Gestión de catálogo (altas, bajas, modificaciones de libros y materiales).
            - Gestión de usuarios (registro, modificación, baja de socios).
            - Procesos de préstamo y devolución (incluyendo reservas y renovaciones).
            - Cálculo y gestión de multas por retraso.
            - Módulo básico de reportes (libros más prestados, usuarios con multas, etc.).
        -

        - **Fuera del Alcance (Excluye):**
            - **Módulo de Contabilidad:** El sistema no gestionará las finanzas de la biblioteca, ni pagos a proveedores, ni facturación electrónica.
            - **Integración con el Sistema Financiero de la Universidad:** No se desarrollarán APIs para conectar con el software de contabilidad externo.
            - **Compra de Libros:** El sistema no incluirá un flujo de trabajo para la adquisición de nuevo material bibliográfico.
    - **Fenómeno Asociado (Scope Creep):** Si esta definición no es clara y aceptada por todos, el proyecto sufrirá de *scope creep* (deslizamiento del alcance). El cliente, viendo el sistema funcionando, pedirá "una pequeña función adicional" que, acumulada con otras, termina desbordando el presupuesto y el cronograma.
2. **Requerimientos No Funcionales: Los Habilitadores Silenciosos del Éxito**
    - **Descripción:** Mientras que los requerimientos funcionales describen *qué* hace el sistema, los no funcionales describen *cómo* lo hace. Son los atributos de calidad del sistema. Frecuentemente subestimados al inicio, su incumplimiento es una de las principales causas de fracaso en producción.
    - **Ejemplos Detallados:**
        - **Rendimiento (Tiempo de Respuesta):** "El sistema debe cargar la página de resultados de búsqueda en menos de 2 segundos con 1000 usuarios concurrentes".
        - **Seguridad:** "El acceso a los datos personales de los usuarios debe requerir autenticación de doble factor. Todos los passwords deben almacenarse en la base de datos utilizando un algoritmo de hash seguro (ej. bcrypt)".
        - **Disponibilidad (Uptime):** "El sistema debe tener una disponibilidad del 99.5% durante el horario de apertura de la biblioteca (8 am - 10 pm)".
        - **Mantenibilidad:** "El código fuente debe estar comentado y estructurado siguiendo las guías de estilo de la empresa, para permitir que un desarrollador que no sea el autor pueda comprender y modificar un módulo en un máximo de 2 días".
        - **Escalabilidad:** "La arquitectura debe permitir que el sistema pueda manejar un crecimiento del 50% en el número de usuarios y transacciones sin requerir un rediseño completo".
    - **Importancia:** Un sistema puede cumplir con todos los requisitos funcionales (hacer préstamos, registrar usuarios, etc.), pero si tarda 30

segundos en cada pantalla o se cae cada dos por tres, será un fracaso rotundo a ojos del usuario.

3. **Factibilidad: El Análisis de Viabilidad**
    - **Descripción:** Antes de comprometer recursos significativos, es imperativo realizar un estudio de factibilidad que responda a preguntas críticas:
        - **Factibilidad Técnica:** ¿Existe la tecnología para construirlo? ¿Tenemos el conocimiento y la experiencia necesarios en el equipo? ¿Podemos integrarlo con los sistemas existentes?
        - **Factibilidad Económica:** ¿El costo de desarrollo y mantenimiento está justificado por el retorno de la inversión (ROI) esperado? ¿Tenemos el presupuesto para completarlo?
        - **Factibilidad Temporal:** ¿Podemos construirlo en el plazo requerido por el negocio o por una oportunidad de mercado?
        - **Factibilidad Operativa:** ¿La organización está preparada para usar y mantener el sistema? ¿Los usuarios finales lo adoptarán?
    - **Ejemplo de Inviabilidad:** Un equipo sin experiencia en machine learning se compromete a construir un sistema de recomendación con inteligencia artificial avanzada para una biblioteca, con un plazo de 3 meses y un presupuesto ajustado. La probabilidad de fracaso es altísima.

**Conclusión sobre PRODUCT**

Definir mal el producto es sembrar el fracaso desde la génesis del proyecto. Una comprensión superficial del "qué" lleva a estimaciones erróneas, planificaciones irreales y, en última instancia, a un producto que no satisface las necesidades del usuario o no es viable técnica o económicamente.

---

### 3️⃣ PROCESS – PROCESO: El Camino a Seguir

**Idea Central: El Marco para la Acción**

Si el Producto define el "qué", el Proceso define el **"cómo"** . Es el marco estructurado de actividades, acciones y tareas que guían el trabajo del equipo para construir el software. Es crucial entender la diferencia fundamental entre **Proceso** y **Proyecto**:

- **Proceso:** Es el conjunto organizado de actividades, principios y prácticas que una organización adopta para desarrollar software de manera consistente. Es genérico y reutilizable.
- **Proyecto:** Es una instancia concreta y única que ejecuta (o adapta) el proceso para construir un producto específico, con un equipo, presupuesto y plazo determinados.

**¿Por Qué es Importante el Proceso?**

Un proceso bien definido aporta múltiples beneficios:

- **Reduce la Improvisación:** Proporciona una hoja de ruta, evitando que el equipo actúe de manera reactiva y desorganizada.
- **Permite el Control y la Visibilidad:** Establece hitos, entregables y puntos de control (ej. revisiones, auditorías) que permiten a los gestores medir el progreso y la calidad.
- **Define Entregables:** Especifica qué productos de trabajo se deben generar (documentos de requisitos, diagramas de diseño, código, casos de prueba) y con qué nivel de calidad.
- **Establece Validaciones:** Define cómo y cuándo se verificará que el trabajo se está haciendo correctamente y se validará que se está construyendo el producto correcto.

**Tipos de Procesos o Ciclos de Vida**

La elección del proceso no es única ni universal. Depende del contexto del proyecto. Pressman clasifica los modelos de proceso en:

- **Modelo en Cascada (Waterfall):** Secuencial y rígido. Ideal para proyectos con requisitos muy estables y bien comprendidos desde el inicio.
- **Modelo Incremental:** Combina secuencia lineal con iteración. Entrega el producto en partes (incrementos) funcionales.
- **Modelos Evolutivos (Iterativos):** Como el desarrollo en espiral o el desarrollo ágil (Scrum, XP). Aceptan y gestionan el cambio como parte natural del proceso. Ideales para entornos de alta incertidumbre y requisitos volátiles.
- **Modelo en Espiral:** Enfocado en la gestión de riesgos. Cada iteración implica identificar y resolver los riesgos más críticos.

**⚠ Error Común: Elegir el Proceso Inadecuado**

Uno de los errores de gestión más graves es imponer un proceso que no se adapta a la naturaleza del proyecto.

- **Ejemplo Catastrófico:** Aplicar un modelo de cascada rígido a un proyecto donde los requisitos son altamente cambiantes (ej. una startup definiendo un nuevo producto).
- **Resultado:** El equipo pasa meses definiendo y documentando requisitos que quedarán obsoletos antes de empezar a codificar. El coste de incorporar el cambio al final del ciclo es prohibitivo, generando frustración y, probablemente, el fracaso del proyecto.

**Conclusión sobre PROCESS**

El proceso no es una camisa de fuerza, ni una solución mágica que elimina los problemas. Es una herramienta de gestión que, bien elegida y adaptada, proporciona la estructura necesaria para que el equipo pueda navegar la complejidad y gestionar los problemas de forma predecible y controlada.

---

## 4️⃣ PROJECT – PROYECTO: La Gestión en Ejecución

**Idea Central: La Ingeniería de la Planificación y el Control**

Aquí es donde convergen las tres P anteriores y donde aparece la gestión formal del día a día. Gestionar el proyecto es el arte de aplicar conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para cumplir con sus requisitos. Abarca desde la planificación inicial hasta el seguimiento y control continuo.

**¿Qué Implica Gestionar el Proyecto? (Las Funciones Clásicas)**

1. **Planificación: El Mapa de Ruta**
    - **Descripción:** La planificación consiste en descomponer el trabajo futuro en unidades manejables y ordenarlas en el tiempo.
    - **Herramientas Clave:**
        - **WBS (Work Breakdown Structure - Estructura de Desglose del Trabajo):** Es una descomposición jerárquica del trabajo total a realizar por el equipo, orientada a los entregables. Su objetivo es dividir un proyecto complejo en partes más pequeñas y fáciles de estimar y gestionar. Por ejemplo, para el sistema de biblioteca, el WBS podría tener ramas como "Módulo de Usuarios", "Módulo de Préstamos", "Documentación", "Pruebas", etc., cada una subdividida en tareas más pequeñas.
        - **Identificación de Dependencias:** Determinar qué tareas deben hacerse antes que otras (ej. no se puede codificar la interfaz de préstamos sin antes haber definido la estructura de la base de datos).
        - **Definición de Hitos (Milestones):** Puntos significativos en el cronograma que marcan la finalización de un entregable importante o una fase clave (ej. "Aprobación del Diseño", "Primera Versión Funcional", "Fin de las Pruebas de Integración").
2. **Estimación: El Arte de Predecir lo Impredecible**
    - **Descripción:** Estimar es el proceso de predecir, con un nivel de incertidumbre asociado, el esfuerzo, el tiempo y el costo necesarios para completar las tareas del proyecto.
    - **Variables a Estimar:**
        - **Tiempo:** Duración del calendario (ej. semanas).
        - **Esfuerzo:** Horas-persona necesarias.
        - **Costos:** Gastos asociados (salarios, herramientas, licencias, infraestructura).
        - **Recursos:** Personas, equipos, software necesario.

- **Importancia Crítica:** Una estimación incorrecta (especialmente por optimismo) es una de las principales causas de fracaso. Un proyecto mal estimado está condenado a exceder su presupuesto y plazo desde el día uno.
- **Herramientas y Técnicas:** Más adelante se estudian técnicas como **Puntos de Función**, modelos de estimación empírica (COCOMO), o la planificación por poker en metodologías ágiles.

3. **Gestión de Riesgos: Prever lo Inesperado**
   - **Descripción:** La gestión de riesgos es un proceso proactivo y continuo para identificar, analizar, planificar la respuesta y monitorear los riesgos que pueden amenazar el éxito del proyecto.
   - **Categorías de Riesgos:**
     - **Riesgos Técnicos:** Problemas con la tecnología, herramientas, rendimiento, calidad.
     - **Riesgos Humanos:** Rotación de personal, conflictos, falta de habilidades, problemas de comunicación.
     - **Riesgos Organizacionales:** Cambios en las prioridades de la empresa, falta de apoyo de la dirección, reestructuraciones.
     - **Riesgos de Herramientas:** Fallos del software de gestión, pérdida de datos, problemas con el entorno de desarrollo.
   - **Actividades Clave:** Identificación de riesgos, análisis de su probabilidad e impacto, planificación de contingencias y monitoreo continuo.
4. **Seguimiento y Control: El Pilotaje Continuo**
   - **Descripción:** Un proyecto no se gestiona solo al inicio con un plan. Requiere un monitoreo constante para comparar el progreso real con el planificado y tomar medidas correctivas cuando sea necesario.
   - **Qué se Monitorea:**
     - **Desviaciones:** Respecto al cronograma y al presupuesto.
     - **Retrasos:** En tareas críticas que pueden afectar la fecha de entrega.
     - **Cambios:** Solicitudes de cambio en el producto, evaluando su impacto.
     - **Calidad:** Número de defectos encontrados, resultados de las pruebas.
   - **Herramientas:** Diagramas de Gantt, tableros Kanban, informes de estado, reuniones de seguimiento.

🎯 **Conclusión sobre PROJECT**

La gestión del proyecto no garantiza el éxito por sí sola, pero su ausencia o deficiencia garantiza, con alta probabilidad, el fracaso. Planificar, estimar, gestionar riesgos y hacer seguimiento son las herramientas que permiten transformar la incertidumbre inherente al desarrollo de software en un riesgo gestionable.

---

### 🔁 Interrelación de las 4 P: Un Sistema Dinámico

Es fundamental entender que las 4 P no son compartimentos estancos. Funcionan como un sistema dinámico donde un cambio en una de ellas provoca un efecto dominó en las demás. Gestionar un proyecto es, en gran medida, gestionar estas interrelaciones.

- **Si cambia el PRODUCTO** (ej. el cliente pide una funcionalidad nueva y compleja) → **cambia el PROYECTO.** Se debe revisar la WBS, re-estimar el esfuerzo, actualizar el cronograma y analizar el impacto en el presupuesto.
- **Si cambia el equipo (PEOPLE)** (ej. se va el arquitecto de software) → **cambia el PROCESO.** Se podría necesitar más tiempo para revisiones de código, aumentar la documentación para transferir conocimiento, o incluso ralentizar el ritmo de desarrollo mientras se forma al nuevo miembro.
- **Si cambia el PROCESO** (ej. se decide adoptar Scrum a mitad de proyecto para manejar mejor los cambios) → **impacta en tiempos y riesgos.** Se necesitarán nuevas reuniones (daily, sprint planning), nuevos roles (Scrum Master), y se redefinirá la forma de entregar valor.
- **Si cambia el PROYECTO** (ej. se adelanta la fecha de entrega por una decisión de negocio) → **afecta motivación y estructura.** El equipo puede sufrir estrés (People), se podrían recortar funcionalidades (Producto), o se podría saltar algunas fases del proceso (Proceso), aumentando el riesgo técnico.

---

### 🎓 Cierre Conceptual para la Clase

La gestión de proyectos de software no es una tarea administrativa secundaria ni un mal necesario. Es la **columna vertebral** que permite transformar una idea abstracta en un sistema de software real, funcional y valioso. Las 4 P de Pressman nos ofrecen un lente para analizar la complejidad de esta tarea de manera estructurada, recordándonos que:

- El éxito se construye sobre **personas** motivadas y competentes.
- Que deben construir un **producto** bien comprendido y definido.
- Siguiendo un **proceso** adecuado a su contexto.
- Y todo ello orquestado mediante una gestión de **proyecto** rigurosa y adaptativa.

Este es el punto de partida. En las próximas clases, profundizaremos en cada una de estas dimensiones, explorando temas como la organización de equipos, la ingeniería de requerimientos, los distintos ciclos de vida, las técnicas de planificación con Gantt y camino crítico, la gestión de riesgos, las métricas y la calidad. Pero recuerden, todo parte de esta visión integradora: **Las 4 P.**